ЦСО: Agent Coding — подход, инструменты и процесс

Материал для самостоятельного изучения. Он объясняет, как с помощью AI-агента превратить идею бизнеса в готовый код и как устроить репозиторий, чтобы агент работал надёжно.

Все примеры вымышленные. Мы показываем подход, а не конкретные сервисы.

> Этот материал собран тем же процессом, который описывает.

Как пользоваться

- **Для кого.** Для инженеров, которые встраивают AI-агента в ежедневную работу — чтобы получать предсказуемый результат, а не «поиграть».
- **Как читать.** По порядку: сначала проблема и идея конвейера, потом фазы, потом архитектура. Схемы — опора, текст — пояснение.
- **Что забрать.** Модель из трёх уровней, шпаргалку «фаза → скилл → артефакт» и минимальный план старта.

Проблема: наивный «промпт → код»

> Один промпт ломается предсказуемо — тремя разными способами.

#	Провал	Что видим
1	**Дрейф терминов и допущений**	Ты говоришь «клиент», агент пишет `account`. Он придумывает поля и связи, которых нет в модели.
2	**Код не там и не той формы**	Логика висит на эндпоинте. Бизнес-правила размазаны по слоям. Диф нечитаем, его нельзя ревьюить.
3	**Нет проверки**	Агент пишет «Готово», а задача не сделана: тесты не гоняли, ревью не было.

Дальше каждая фаза конвейера закрывает свой провал. Дрейф лечим контекстом. «Не там» — архитектурой. «Нет проверки» — процессом.

Главная идея

Конвейер фаз. Agent coding — это конвейер фаз, а не один длинный промпт. На выходе каждой фазы — готовый артефакт.

- Каждая фаза работает по схеме «вход → выход».
- Каждая фаза закрывает свой класс провалов.
- Повторить фазу дёшево. Переписать всё с нуля — дорого.

Архитектура как интерфейс. Архитектура и документы репозитория — это интерфейс агента. Без них конвейер упирается в неоднозначность.

Уровни конвейера

> Идея проходит три уровня — от бизнеса до кода.

- **Уровень 1 — Идея.** Бизнес пишет задачу-эпик в трекере задач.
- **Уровень 2 — Планирование.** Команда планирует задачи в структуре и языке наших сервисов.
- **Уровень 3 — Реализация.** Команда детально прорабатывает задачу внутри каждого сервиса.

Чем больше номер уровня, тем ближе к коду. Один **spec** уровня 2 запускает **N** прогонов уровня 3 — по одному на каждый затронутый сервис.

Уровень 1 — Идея

Эпик в трекере

> Бизнес пишет задачу-эпик в трекере задач.

Бизнес формулирует идею как эпик в трекере задач. Эпик — точка входа: одна крупная задача, часто на несколько сервисов.

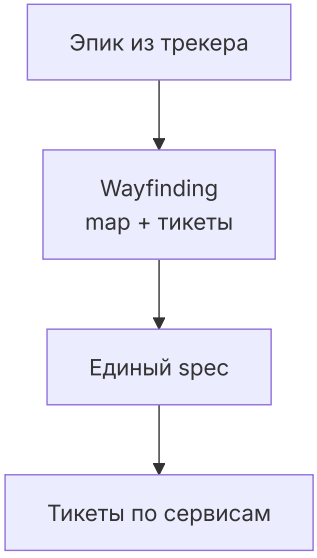
- Эпик живёт в трекере, а не в репозитории.
- В нём — суть идеи от бизнеса, без технических деталей.
- На этом уровень 1 заканчивается. Дальше эпик берёт в работу команда.

Артефакт уровня: эпик в трекере задач.

Уровень 2 — Планирование

> Команда берёт эпик и планирует задачи в структуре и языке наших сервисов.

Работу ведём в док-хабе — репозитории с описаниями и глоссариями всех сервисов. Кода в нём нет. Здесь эпик превращается в единый spec и тикеты по сервисам.



Док-хаб

Док-хаб объединяет несколько сервисов. В корне лежит `CONTEXT-MAP.md`, он ведёт к `CONTEXT.md` каждого сервиса. Глоссарии под рукой — описываем задачи в языке сервисов.

Что это даёт:

- Задача лежит в одном месте и не расползается по чатам.
- Терминология всех сервисов всегда под рукой.
- У работы есть папка и история. Ничего не теряется.

Wayfinding: планируем картой

> Большую идею не разобрать за одну сессию — поэтому мы планируем её картой.

Шаги:

1. Заводим карту — `.scratch/<effort>/map.md`.
2. Режем идею на исследовательские тикеты.
3. Резолвим тикеты по одному.
4. Копим ответы в «Decisions so far». Туман превращаем в явные вопросы.

Артефакт:

```
.scratch/<effort>/
├── map.md        ← карта проработки
└── issues/
```

```
    ├── 01-....md ← исследовательские тикеты
    └── 02-....md
```

Локальный markdown работает как трекер: его ревьюят, и он живёт в git.

```
# Map: Интеграция Поручений и Обращений

## Destination
Единый spec + тикеты по сервисам (appeal, services, fo, partners, tasks, ...)

## Notes
- Контексты: src/services, src/appeals, src/fo, ...
- Репозитории: concierge-appeal, sbpv_services, ...

## Decisions so far
- [T01: Research] — сводка схем и контрактов по 7 репозиториям
- [T02: RMQ-события] — appeal.created/updated, идемпотентность через row_hash
- ...

## Not yet specified (fog)
- Формат хендшейка dzo_appeal_id при смене типа

## Out of scope
- Реализация кода — map завершается на spec + тикетах
```

Каждый узел карты — отдельный исследовательский тикет. Мы резолвим их по одному. Так фраза «мы не знаем X» превращается в тикет, а не в сюрприз посреди разработки.

Декомпозиция по сервисам

> Мы режем проработанный эпик на тикеты «сервис × сценарий» — с границей и связями.

За уровень 2 отвечает лид или архитектор. Он ведёт эпик поперёк сервисов. На выходе — тикеты, по одному на каждый сценарий сервиса. У одного сервиса их может быть несколько.

Тикет уровня 2 — это **изменение контракта**, а не изменение кода. Поэтому в нём две части:

- **Внешние рёбра** — чем сервис связан с соседями: API и события. Это видят другие сервисы, отсюда берётся `Blocked by` между сервисами.
- **Внутренние рёбра** — только если у этого сервиса несколько тикетов: изменения схемы БД, которые гейтят следующий тикет. Помечаем их как внутренние, чтобы не приняли за контракт.

Схему БД и структуру модулей в контракт не тащим: соседи читают API и события, а не чужую базу. Внутреннюю структуру додумает уровень 3.

Шаблон:

```
# <Сервис> — <сценарий>
Status: ready-for-agent
Blocked by: <тикеты, без которых нельзя начать>

## Граница
API: <методы + формат запроса/ответа>
События: <топик + payload>

## Поведение (end-to-end, видимое снаружи)

## Внутренние опоры   ← только если у сервиса есть другие тикеты
- Schema: <таблица/поле, которое гейтит следующий тикет>

## Acceptance (интеграционная, проверяется на границе)

## Ссылки
Spec: <секция>  ·  Эпик: <ссылка L1>
```

Два уточнения:

- **Контракт определяет потребитель.** Пишем не «что сервис делает», а «что нужно соседу». Иначе `Blocked by` и acceptance повисают в воздухе.
- **Совместимость — решение уровня 2.** Аддитивное изменение или ломающее влияет на соседей, значит решаем здесь, а не в L3.

Тикет готов, когда репозиторий может запустить свой конвейер уровня 3 и сам додумать внутреннюю структуру.

Зачем так:

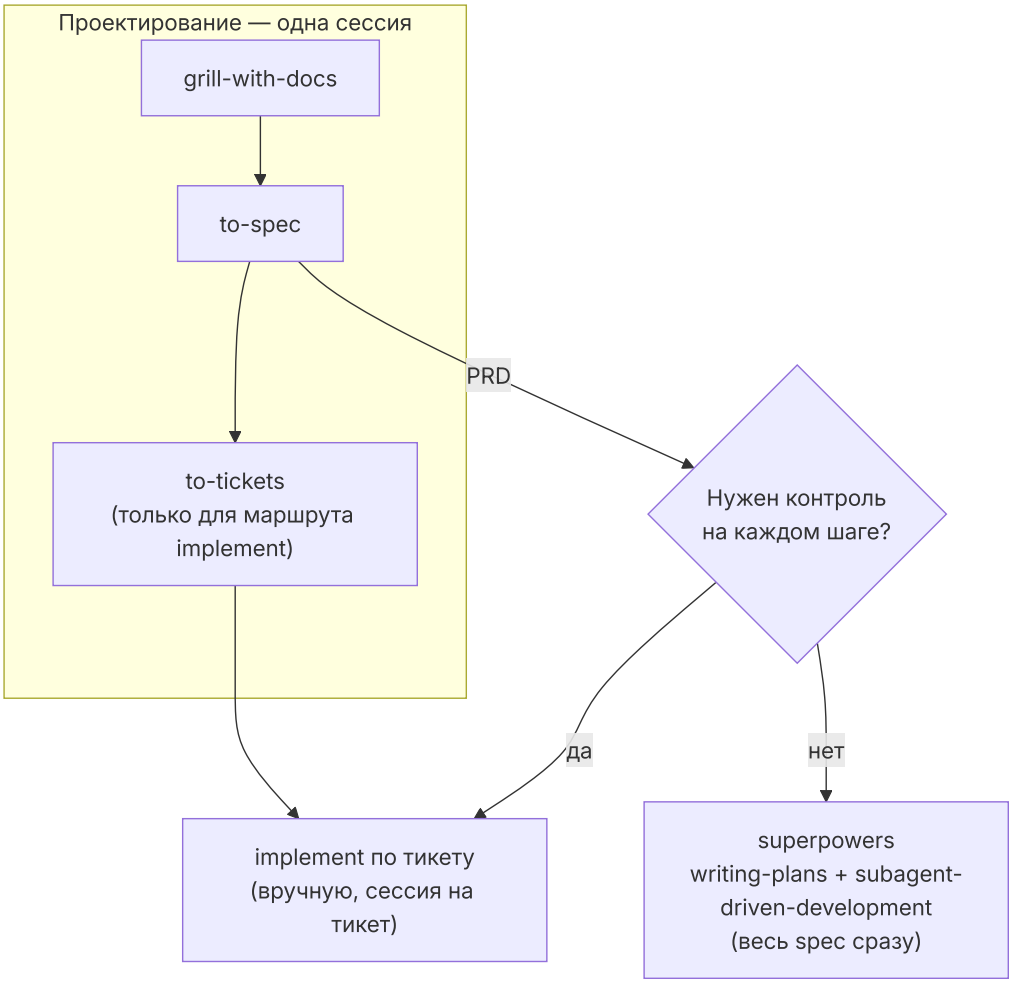
- Тикеты можно раскидать по разработчикам и агентам.
- `Blocked by` задаёт порядок. Сразу видно, что ничего не потеряли.
- Идея бизнеса трассируется до конкретных изменений.

На этом уровень 2 заканчивается. На выходе — единый spec и набор тикетов с блокерами. Дальше каждый сервис идёт по своему конвейеру — это уровень 3.

Уровень 3 — Реализация

Две стадии, и их не смешиваем.

- **Проектирование** — одна сессия: `grill-with-docs` → `to-spec` → `to-tickets`. Тикеты нужны только маршруту через `implement`.
- **Исполнение** — развилка на два маршрута.



Артефакт прошлой фазы — вход для следующей. Поэтому фазы перезапускаемы: изменился PRD — заново собрали тикеты, не переписывая всё.

Проектирование — одна сессия

Здесь рождаются PRD и тикеты. Проектирование не смешиваем с исполнением: это одна сессия, и она заканчивается до первой строки кода.

`grill-with-docs`

> `grill-with-docs` — это чат с архитектором: ты ведёшь процесс и принимаешь решения, ИИ прорабатывает техническую реализацию и фиксирует требования.

Скилл берёт интервью по одному вопросу и попутно пишет глоссарий и ADR. Состав: `grilling` + `domain-modeling`.

Шаги:

1. Агент задаёт один вопрос. К каждому вопросу даёт свою рекомендацию.
2. Идём по дереву решений. Снимаем зависимости по одной.
3. Агент сам ищет факты в коде и сверяет с ним утверждения. Решения оставляет тебе.
4. Туманные термины уточняем. Конфликт с глоссарием вскрываем сразу. Крайние случаи проверяем сценариями.
5. Агент пишет `CONTEXT.md` и ADR прямо по ходу сессии.

Пока мы не договорились, план не реализуем.

Артефакты:

```
# Orders

## Language

**Order**:
A request from a Customer to buy goods.
_Avoid_: Purchase, transaction

**Invoice**:
A request for payment sent after delivery.
_Avoid_: Bill, payment request
```

```
# Docs split by moment

Rules were duplicated between architecture.md
and CODING_STANDARDS.md → drift.
Split by the moment in the agent cycle;
each rule lives in exactly one document.
```

Глоссарий — рабочая вещь, а не «документация на будущее». Он не даёт агенту дрейфовать в синонимы на следующих фазах. ADR пишем только тогда, когда решение трудно откатить, оно удивительно без контекста и у него была реальная альтернатива.

`to-spec`

> Скилл собирает спеку из уже сказанного — без нового интервью.

Скилл не спрашивает. Он синтезирует то, что мы уже обсудили.

Шаги:

1. Агент изучает репозиторий и берёт термины из глоссария.
2. Соблюдает ADR затронутой области.
3. Намечает швы для тестирования: сначала ищет существующие, а если их нет — выбирает самый высокий уровень, на котором можно изолированно проверить сценарий. В идеале достаточно одного шва. Сверяет выбор с тобой.
4. Пишет спеку по шаблону: без путей к файлам и сниппетов.
5. Публикует сразу с меткой `ready-for-agent`.

Шаблон:

```
## Problem Statement
## Solution
## User Stories          ← длинный список
## Implementation Decisions
## Testing Decisions
## Out of Scope
## Further Notes
```

Ключевые секции — **Out of Scope** и **Testing Decisions**. Они не дают агенту «додумать» лишнее.

`to-tickets` — только для маршрута implement

> Спека превращается в тонкие вертикальные срезы с явными блокерами.

Тикеты нужны маршруту с ручным контролем. Второй маршрут (superpowers) работает по PRD напрямую.

Что такое тикет.

- **Вертикальный срез.** Узкий, но полный путь через все слои: схема, API, тесты. Не срез одного слоя.
- Готовый срез можно проверить и показать сам по себе.
- Размер — под одно свежее контекстное окно.
- Сначала **prefactoring**: сделай изменение лёгким, потом внеси лёгкое изменение.
- **Wide refactor** — исключение. Его ведут как expand → migrate (батчами) → contract.

Шаги.

1. Агент собирает контекст.
2. Изучает код и ищет, что отрефакторить заранее.
3. Чертит вертикальные срезы.
4. Проставляет блокеры.
5. Показывает список и спрашивает: гранулярность? блокеры? что объединить или разбить?
6. Публикует тикеты после согласования.

Шаблон тикета (`.scratch/<feature>/issues/NN-*.md`):

```
# 03: Offer readiness rule

**What to build:** оффер нельзя отправить,
если у него нет ни одного активного атрибута.

**Blocked by:** 02
**Status:** ready-for-agent

- [ ] ...
```

Нумерация идёт с `01` в порядке зависимостей.

Исполнение — два маршрута

Выбор — по контролю. Проектирование закончено, у нас на руках PRD и (для маршрута 1) тикеты.

Маршрут 1 · Ручной контроль

`implement` запускается **вручную по каждому тикету** — отдельной сессией. Свежая сессия — свежее контекстное окно.

- Внутри — TDD: сначала тест, потом код.
- Контроль на каждом шаге: смотришь и правишь.
- По завершении `implement` сам зовёт `code-review`.
- Найденное правишь сразу — в код или в тикет.

Маршрут 2 · superpowers: весь spec сразу

PRD уходит в конвейер superpowers. Сначала `writing-plans` пишет план и режет его на **мелкие шаги** (один шаг — 2–5 минут): написать падающий тест → прогнать, убедиться что падает → минимальная реализация → тесты зелёные → коммит. Каждая задача — с точными файлами, тестами и командами.

Потом `subagent-driven-development` выполняет план **задача за задачей**: на каждую задачу — свежий субагент, после каждой — ревью (соответствие спеке + качество), в конце — общее ревью ветки.

- Контроля меньше, но не нужно запускать каждый тикет вручную.
- Нет переполнения контекста: исполнение и ревью всегда идут **чистым субагентом**.
- Тикеты не нужны — их роль играет план.

Ревью после каждой задачи

> Проверка встроена в исполнение, а не вынесена в конец.

Оба маршрута проверяют работу после каждой задачи.

- **Маршрут 1.** `implement` ведёт задачу по TDD (сначала тест, потом код) и по завершении сам зовёт `code-review`.
- **Маршрут 2.** `subagent-driven-development` даёт свежий субагент на задачу и ревью после каждой; в конце — общее ревью ветки.

Проверки на выходе:

- Гоняем тесты. Не «написали», а прогнали. Typecheck — регулярно, полный прогон — в конце.
- `verification-before-completion` — сначала вывод команд, потом утверждение об успехе.
- `code-review` — ревью по рубрике репозитория: стандарты + соответствие спеке.

Почему после каждой задачи, а не в конце:

- Дефект ловится, пока контекст задачи свеж, а правка дешёвая.
- Ревью по рубрике детерминированное: одна планка для агента и человека.
- Найденный дефект чаще всего указывает на пропущенное решение или термин. Значит, возврат к проектированию, а не к исполнению.

Архитектура как интерфейс агента

> Кодовая база — интерфейс агента, а не просто файлы.

Наивный репозиторий	Репозиторий как интерфейс
О слоях договорились на словах. Код лежит где попало.	У каждого типа кода ровно одно место и одна форма.
Три слова для одного понятия.	Термины зафиксированы в глоссарии.
Решения помнят только люди.	Решения записаны в ADR вместе с «когда ломать».
Агент угадывает — и каждый раз по-разному.	Агент находит место, а не изобретает его.

Провал №2 («код не там и не той формы») — следствие архитектуры, а не промпта. Архитектуру лечим документами и правилами, а не просьбами к агенту.

Почему DDD

> Агент не держит систему в голове — только контекстное окно.

Весь сервис в это окно не влезает. Значит, архитектуру режем на куски, которые агент удержит по отдельности, а не описываем целиком. Тут DDD выигрывает: у каждого понятия одно имя, у задачи есть конечный объём, а домен не зависит от базы — тесты идут без моков. Так команда использует общепринятые понятия DDD и не изобретает собственную терминологию.

Что берём из DDD

> Из DDD берём три вещи: единый язык, границы сервисов и изоляцию домена.

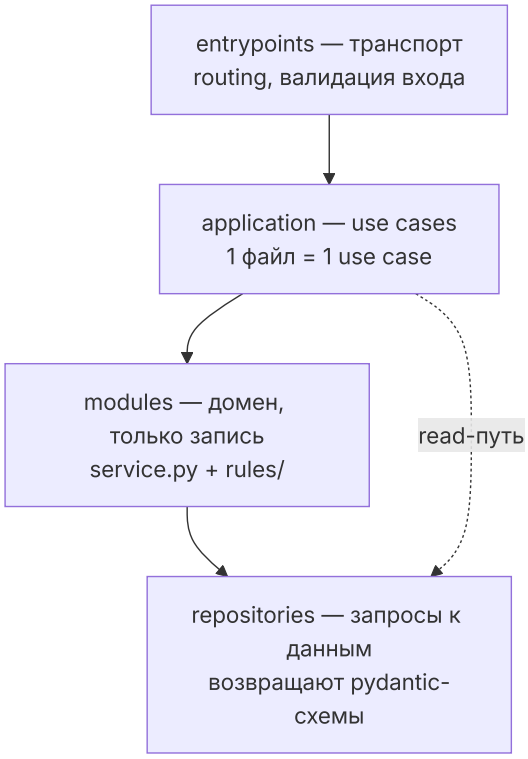
- **Ubiquitous language.** Один словарь на сервис. Мы фиксируем его в глоссарии (`CONTEXT.md`). Человек и агент называют вещи одними словами. *Избегаем*: терминология, naming convention.
- **Bounded context + Context Map.** Один сервис — один глоссарий. `CONTEXT-MAP.md` описывает, какие сервисы есть и как они связаны. *Избегаем*: модуль, сервис.
- **Domain isolation.** Домен не зависит от транспорта и базы. Мы тестируем его отдельно и переиспользуем. *Избегаем*: бизнес-логика, domain layer.

От «канонического» DDD отличие минимально. Мы берём те части, которые становятся контрактом для агента: язык, границы, изоляция.

Слои и направление зависимостей

> У каждого типа кода ровно одно место.

Слои не советуют. Они задают, куда что положить и что откуда импортировать.



- `application` только ведёт сценарий и работает асинхронно. Один файл — один use case: `create_offer.py` — это сценарий `create_offer`. Новый сценарий — новый файл, а не ветка в старом.
- `modules` — единственный, кто пишет. `service.py` пишет в базу, но не ведёт сценарий.
- `rules/` — чистые sync-функции без базы и сети (`is_*`, `validate_*`, `resolve_*`).
- Чтение идёт мимо `modules`: `entrypoints → application → repositories`.
- `application` не импортирует `entrypoints`.
- `modules` не импортирует `application`.
- Домен не бросает ошибки транспорта (`HTTPException`).

Правило слоёв превращает «где это место?» из творческого вопроса в поиск по таблице. Агент больше не вешает логику на эндпоинт. А «один файл — один use case» делает то же на уровне задачи: тикет называет файл, правки касаются одного сценария, и весь слой читать не нужно.

Глоссарий и ADR

> Глоссарий фиксирует язык. ADR фиксирует решения и причину.

`CONTEXT.md` — глоссарий. Только язык, никакой реализации.

- Один канонический термин на понятие.
- Список `_Avoid_` — синонимы, которых не используем.

```
**Invoice**:
A request for payment sent after delivery.
_Avoid_: Bill, payment request
```

`docs/adr/` — решения. Пишем ADR, только когда сходятся все три условия:

1. Решение трудно откатить.
2. Без контекста оно удивляет («зачем они так?»).
3. Мы выбирали между реальными альтернативами.

ADR — один абзац. Ценность в факте: «решили так и вот почему». Оба документа заводим, только когда термин или решение реально появились.

`architecture.md` — как писать код

> Перед тем как писать код, агент читает таблицу «куда что класть».

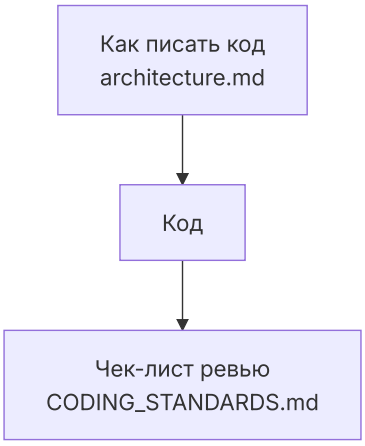
Ты пишешь...	Клади в	Как назвать

Ты пишешь…	Клади в	Как назвать
Разбор входа (HTTP body → типизированное значение)	`schemas/application/*`	`XxxSchema`
Чистая функция над состоянием (без IO)	`modules/<entity>/rules/<concept>.py`	`is_*`, `validate_*`, `resolve_*`, `derive_*` — sync `def`
Запись в базу + история	`modules/<entity>/service.py`	`create`, `update`, `upsert`, `set_<child>`
Ведёшь сценарий, проверяешь данные с базой	`application/<verb>_<entity>.py`	`create_*`, `get_*` — `async def`
Запросы к данным	`repositories/*`	`get_list`, `get_by_id`, `create`, `update`, `delete`

В каждом документе есть секция **«когда ломать правило»** и причина. Агент не просто следует правилу. Он понимает, где правило не работает, и говорит об этом. Молча не ломает.

`CODING_STANDARDS.md` — чек-лист ревью

> Тот же свод правил, но для ревью. Каждое правило живёт ровно в одном документе.



- `architecture.md` агент читает перед тем, как писать код.
- `CODING_STANDARDS.md` — при ревью.
- Один источник на правило. Второй документ ссылается, а не повторяет.

Так документы не повторяют друг друга и не расходятся: «как пишем» и «как ревьюим» совпадают. На каждой фазе агент читает только нужный документ — меньше контекста. Плата за это — документ не самодостаточен. Мы выбрали один источник на правило.

Модели и безопасный контур

> Модель доступна агенту только через корпоративный контур: напрямую к провайдеру он не обращается.

Для agent coding используем **DeepSeek V4.1 Flash**. Агент обращается к ней через корпоративный proxy на базе **LiteLLM**, который служит единой точкой доступа к моделям.

На proxy настроены guardrails: перед отправкой запроса в модель они маскируют персональные данные (ПДн), токены, секреты и ссылки на внутренние ресурсы. Это позволяет использовать контекст из рабочих задач, сохраняя чувствительные данные внутри корпоративного контура.

Инструменты

> Скилл — фаза конвейера, записанная как инструкция для агента.

Всё делаем через opencode — одну точку входа: сессии, skills, MCP, права. Через него идут и планирование, и код, и ревью.

- codegraph — граф символов и вызовов. Агент находит нужный код без grep-циклов.
- LiteLLM — основа корпоративного proxy и единый шлюз к моделям. Агент ходит в модели только через него.
- **Локальный markdown-трекер** — `.scratch/` живёт в git: историю правок видно, её можно ревьюить.

Скиллы как фазы конвейера:

- Matt Pocock skills — `grill-with-docs`, `to-spec`, `to-tickets`, `implement`, `wayfinder`, `code-review`, `triage`.
- superpowers — `brainstorming`, `writing-plans`, `subagent-driven-development`, `verification-before-completion`.

Шпаргалка: фаза → скилл → артефакт

Уровень	Фаза	Скилл	Артефакт на выходе
1	Эпик в трекере	—	эпик в трекере задач
2	Wayfinding	`wayfinder`	`map.md` + исследовательские тикеты
2	Декомпозиция	`to-tickets`	тикеты «сервис × сценарий» с `Blocked by`
3	Проектирование	`grill-with-docs` (+ `domain-modeling`)	`CONTEXT.md`, `docs/adr/*`
3	Спека	`to-spec`	`PRD.md`
3	Тикеты (только для implement)	`to-tickets`	`.scratch/<feature>/issues/NN-*.md`
3	Исполнение · маршрут 1	`implement` по тикету	код
3	Исполнение · маршрут 2	`writing-plans` + `subagent-driven-development`	план + код
3	Ревью после задачи	`code-review`, `verification-before-completion`	зелёные тесты, ревью

Всё идёт через **opencode**. Понимание кода — **codegraph**. Модель — **DeepSeek V4.1 Flash** через корпоративный proxy на **LiteLLM** с guardrails. Трекер — markdown в **`.scratch/`**.

С чего начать

> Сначала процесс, потом мощность.

1. **Введи глоссарий** (`CONTEXT.md`). Это самый дорогой по эффекту шаг. Без единого языка конвейер рассыпается на дрейфе.
2. **Раздели доки по моменту.** Гид для написания и чек-лист ревью. Одно правило живёт ровно в одном месте.
3. **Напиши `architecture.md`.** Таблица «куда что кладётся» + секция «когда ломать правило».
4. **Пройди один реальный прогон** на небольшой фиче: grill → spec → tickets → implement → verify.
5. **Подключи субагентный конвейер** только после этого.

Сопротивление возникает на шагах 1–3. Это рутина, которая «не пишет код». Но именно она делает шаги 4–5 предсказуемыми.

Итог

1. **Конвейер, а не промпт.** Фазы с готовым артефактом закрывают предсказуемые провалы.
2. **Три уровня.** Уровень 1–2 разводят идею по сервисам. Уровень 3 доводит её до кода.
3. **Архитектура — интерфейс агента.** Слои, глоссарий, ADR и golden reference — это контракт, а не украшение.
4. **Один источник на правило.** Доки делятся по моменту цикла. Правило не дублируется.
5. **Ревью — после каждой задачи.** И TDD-исполнение, и ревью идут на каждом шаге, а не только в конце.

Полезные ссылки

- opencode — среда, в которой идёт вся работа: сессии, skills, MCP, права.
- codegraph — граф символов и вызовов вместо grep-циклов.
- LiteLLM — основа корпоративного proxy и единого шлюза к моделям; guardrails маскируют ПДн, токены, секреты и ссылки на внутренние ресурсы.
- Matt Pocock skills — `grill-with-docs`, `to-spec`, `to-tickets`, `implement`, `wayfinder`, `code-review`, `triage`.
- superpowers — `brainstorming`, `writing-plans`, `subagent-driven-development`, `verification-before-completion`.